

Lab_W7: Xây dựng & Huấn luyện MLP + Hyperparameter Tuning

Triết lý lab: Không có bước nào là “gỡ theo cho xong”. Mỗi mốc quyết định đều kèm một câu hỏi “Vì sao?” phải trả lời được trước khi chạy ô code tiếp theo. Dataset tổng hợp (không cần tải gì), framework chính là PyTorch.

Cách làm: Chạy lần lượt từng Phase. Mọi câu hỏi đánh dấu **?** là **bắt buộc** trả lời trong báo cáo, mỗi câu một đoạn ngắn giải thích lý do, không chỉ chép kết quả.

Bản đồ tổng thể

Hiểu bài toán → Chuẩn bị data ĐÚNG → Baseline tối giản
→ Vòng lặp train + ĐỌC đường loss → Đánh giá trung thực
→ Tuning CỐ HỆ THỐNG → Chẩn đoán khi hỏng

Quy tắc xuyên suốt: **làm cho nó chạy (dù tệ) trước, rồi mới làm cho nó tốt.**

Phase 0 — Môi trường & cố định ngẫu nhiên

```
import numpy as np, torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
SEED = 42
np.random.seed(SEED); torch.manual_seed(SEED)
device = "cuda" if torch.cuda.is_available() else "cpu"
```

? Vì sao cố định seed? Nó cho kết quả “đáng tin hơn” hay chỉ “lặp lại được”? (Hai khái niệm này khác nhau.)

Phase 1 — Dữ liệu: nơi 80% lỗi thật sự xảy ra

```
X, y = make_classification(n_samples=4000, n_features=20, n_informative=8,
                          n_redundant=4, n_classes=2, class_sep=1.0, flip_y=0.03, random_state=SEED)

# Chia TRƯỚC khi scale
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.15, stratify=y, random_state=SEED)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.1765, stratify=y_temp, random_state=SEED)
# => ~70% train / 15% val / 15% test

# Fit scaler CHỈ trên train
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

? Nếu fit scaler trên TOÀN BỘ X rồi mới chia, rò rỉ cái gì, từ đâu sang đâu? (Đây là data leakage kinh điển.)

? Vì sao cần val TÁCH RỜI test? Nếu chỉ có train/test và tuning theo test thì ta ăn gian vào tập nào?

```
def make_loader(X, y, batch_size=64, shuffle=False):
    ds = TensorDataset(torch.tensor(X, dtype=torch.float32),
                        torch.tensor(y, dtype=torch.float32))
    return DataLoader(ds, batch_size=batch_size, shuffle=shuffle)

train_loader = make_loader(X_train, y_train, 64, shuffle=True)
val_loader = make_loader(X_val, y_val, 256)
test_loader = make_loader(X_test, y_test, 256)
```

? Vì sao shuffle=True cho train nhưng False cho val/test?

Phase 2 — Baseline MLP tối giản (cổ tình “ngu”)

Mục tiêu KHÔNG phải accuracy cao, mà là một đường ống chạy thông data → loss → cập nhật trọng số.

```
class MLP(nn.Module):
    def __init__(self, in_dim, hidden=(32,), p_drop=0.0):
        super().__init__()
        layers, d = [], in_dim
        for h in hidden:
            layers += [nn.Linear(d, h), nn.ReLU(), nn.Dropout(p_drop)]
            d = h
        layers += [nn.Linear(d, 1)] # 1 logit cho nhị phân
        self.net = nn.Sequential(*layers)
    def forward(self, x):
        return self.net(x).squeeze(-1)

model = MLP(in_dim=20, hidden=(32,)).to(device)
```

? Vì sao tầng cuối ra 1 logit và KHÔNG có sigmoid ở đây? (Liên quan tới BCEWithLogitsLoss bên dưới.)

? Vì sao chọn ReLU chứ không sigmoid/tanh cho tầng ẩn?

? Bắt đầu với 1 tầng ẩn 32 neuron dựa trên lý lẽ gì? (Gợi ý: “đơn giản nhất có thể thắng baseline”).

? Dropout đang để 0 — khi nào bật lên, dựa vào tín hiệu nào?

Phase 3 — Vòng lặp huấn luyện + ĐỌC đường loss

```
def train_model(model, train_loader, val_loader, epochs=50, lr=1e-3,
                weight_decay=0.0, patience=8):
    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr,
                                   weight_decay=weight_decay)
    history = {"train": [], "val": []}
    best_val, best_state, wait = float("inf"), None, 0
    for epoch in range(epochs):
        model.train(); tr_loss = 0.0
        for xb, yb in train_loader:
            xb, yb = xb.to(device), yb.to(device)
            optimizer.zero_grad()
            loss = criterion(model(xb), yb)
            loss.backward(); optimizer.step()
            tr_loss += loss.item() * xb.size(0)
        tr_loss /= len(train_loader.dataset)
```

```

model.eval(); va_loss = 0.0
with torch.no_grad():
    for xb, yb in val_loader:
        xb, yb = xb.to(device), yb.to(device)
        va_loss += criterion(model(xb), yb).item() * xb.size(0)
va_loss /= len(val_loader.dataset)
history["train"].append(tr_loss); history["val"].append(va_loss)

if va_loss < best_val - 1e-4:
    best_val, wait = va_loss, 0
    best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}
else:
    wait += 1
    if wait >= patience: break
model.load_state_dict(best_state) # khôi phục trọng số tốt nhất
return history

```

Năm điểm phải giải thích được:

? Trật tự `zero_grad` → `forward` → `loss` → `backward` → `step`: đảo cái nào sẽ sai? Vì sao `zero_grad` mỗi vòng?

? Vì sao gọi `model.train()` rồi `model.eval()`? Cái gì đổi hành vi giữa hai chế độ? (Gợi ý: Dropout, BatchNorm.)

? Vì sao validate bọc trong `torch.no_grad()`?

? Early stopping dùng tập NÀO làm tín hiệu dừng? Vì sao tuyệt đối không dùng test?

? Vì sao lưu `best_state` và khôi phục, thay vì lấy trọng số ở epoch cuối?

Đọc đường loss — bảng chẩn đoán

Hình dạng quan sát	Chẩn đoán	Hành động
Train ↓, Val ↓ rồi đi ngang sát nhau	Học tốt	Có thể tăng nhẹ năng lực mô hình
Train ↓ mạnh, Val ↓ rồi quay đầu đi lên	Overfit	Regularization / dropout / dừng sớm / giảm model
Cả hai cao và không giảm	Underfit hoặc lr sai	Tăng năng lực / chỉnh lr / kiểm tra data
Loss nhảy loạn / NaN	lr quá lớn, hoặc data chưa scale	Giảm lr, kiểm tra Phase 1

? Vẽ `history["train"]` và `history["val"]`; nhóm em rơi vào DÒNG NÀO của bảng? Gọi tên hiện tượng trước khi nói accuracy.

Phase 4 — Đánh giá trung thực (chạm test ĐÚNG MỘT LẦN)

```

from sklearn.metrics import (accuracy_score, f1_score,
                             roc_auc_score, confusion_matrix)

def evaluate(model, loader):
    model.eval(); logits, ys = [], []
    with torch.no_grad():
        for xb, yb in loader:
            logits.append(model(xb.to(device)).cpu()); ys.append(yb)
    logits = torch.cat(logits); ys = torch.cat(ys).numpy()
    probs = torch.sigmoid(logits).numpy()
    preds = (probs >= 0.5).astype(int)
    return {"accuracy": accuracy_score(ys, preds),

```

```
"f1": f1_score(ys, preds),
"auc": roc_auc_score(ys, probs),
"cm": confusion_matrix(ys, preds)}
```

? Vì sao test chỉ chạy MỘT LẦN ở cuối? Và vì sao accuracy có thể “đẹp” mà mô hình vẫn vô dụng? (Gợi ý: mất cân bằng lớp → nhìn F1/AUC/confusion matrix.)

Phase 5 — Hyperparameter Tuning CỐ HỆ THỐNG

Nguyên tắc: tuning là **thí nghiệm có kiểm soát**, không đoán mò. Đi từ thô tới tinh theo 3 nấc.

5.0 — Phân loại siêu tham số

Nhóm	Ví dụ	Ảnh hưởng tới
Năng lực mô hình	số tầng, số neuron, dropout	bias ↔ variance
Tối ưu hóa	learning rate, optimizer, batch size, epoch	tốc độ & độ ổn định hội tụ
Regularization	weight_decay (L2), dropout, patience	chống overfit

Learning rate gần như luôn là siêu tham số đáng tinh chỉnh nhất. Nếu chỉ được chỉnh một thứ, chỉnh nó.

5.1 — Nấc 1: Dò tay (manual)

Chạy 3–4 cấu hình tay với $lr \in \{1e-2, 1e-3, 1e-4\}$, quan sát đường loss để có trực giác về thang giá trị.

? Với mỗi lr, đường loss biểu hiện ra sao? Mô tả, đừng chỉ chọn cái tốt nhất.

5.2 — Nấc 2: Random Search (mặc định, hơn Grid)

```
import random
search_space = {
    "hidden": [(32,), (64,), (64, 32), (128, 64)],
    "lr": [1e-2, 3e-3, 1e-3, 3e-4],
    "weight_decay": [0.0, 1e-4, 1e-3],
    "p_drop": [0.0, 0.2, 0.4],
    "batch_size": [32, 64, 128],
}
N_TRIALS = 15; results = []
for t in range(N_TRIALS):
    cfg = {k: random.choice(v) for k, v in search_space.items()}
    tl = make_loader(X_train, y_train, cfg["batch_size"], shuffle=True)
    m = MLP(20, hidden=cfg["hidden"], p_drop=cfg["p_drop"]).to(device)
    train_model(m, tl, val_loader, lr=cfg["lr"],
                weight_decay=cfg["weight_decay"])
    auc = evaluate(m, val_loader)["auc"]
    results.append((cfg, auc))
best_cfg, best_auc = max(results, key=lambda r: r[1])
```

? CÂU CỐT LỖI: Vì sao Random Search thường tốt hơn Grid Search với cùng ngân sách? (Gợi ý: có bao nhiêu siêu tham số thực sự quan trọng?)

? Ta chọn cấu hình thắng theo metric nào, trên tập nào? Vì sao không phải accuracy, không phải test?

5.3 — Nấc 3: Bayesian Optimization

```
import optuna
def objective(trial):
```

```

n_layers = trial.suggest_int("n_layers", 1, 3)
lr = trial.suggest_float("lr", 1e-4, 1e-2, log=True)
wd = trial.suggest_float("weight_decay", 1e-6, 1e-2, log=True)
pdr = trial.suggest_float("p_drop", 0.0, 0.5)
bs = trial.suggest_categorical("batch_size", [32, 64, 128])
hidden = tuple(trial.suggest_int(f"u{i}", 16, 128, log=True)
               for i in range(n_layers))
tl = make_loader(X_train, y_train, bs, shuffle=True)
m = MLP(20, hidden=hidden, p_drop=pdr).to(device)
train_model(m, tl, val_loader, lr=lr, weight_decay=wd)
return evaluate(m, val_loader)["auc"]

study = optuna.create_study(direction="maximize",
                           sampler=optuna.samplers.TPESampler(seed=SEED))
study.optimize(objective, n_trials=30)

```

? Vì sao lr và weight_decay lấy mẫu trên thang LOG chứ không tuyến tính?

5.4 — Khung chuẩn vàng: Cross-validation

Với dataset vừa/nhỏ, một tập val đơn lẻ rất nhiều. Quy trình nghiêm túc: k-fold CV trên (train+val) để chấm mỗi cấu hình → chốt cấu hình thắng → train lại trên toàn bộ train+val → đánh giá MỘT lần trên test.

? Vì sao CV ổn định hơn một lần chia val? Cái giá là gì? Khi nào CV không đáng?

Phase 6 — Bảng chẩn đoán “khi mô hình hỏng”

Triệu chứng	Nguyên nhân hay gặp	Cách xử lý theo thứ tự
Loss = NaN ngay epoch đầu	lr quá lớn / data chưa scale / có inf	giảm lr 10×, kiểm tra Phase 1, gradient clipping
Train loss không giảm	lr quá nhỏ / model quá yếu / sai nhãn	tăng lr, tăng năng lực, kiểm tra nhãn
Train tốt, Val tệ dần	Overfit	dropout↑, weight_decay↑, dừng sớm, giảm model
Val nhảy loạn giữa epoch	batch quá nhỏ / lr cao / val nhỏ	tăng batch, giảm lr, dùng CV
Test << Val	Đã leak / tuning quá nhiều vào val	rà lại Phase 1, giảm số lần nhìn val

Nguyên tắc gỡ lỗi vàng: mỗi lần chỉ đổi MỘT thứ. Đổi 3 thứ rồi tốt lên thì không biết thứ nào tạo ra điều đó.

Checklist nộp bài

Mỗi SV cần nộp, với mỗi mục là **một câu lý do**:

- ☐ Tỷ lệ chia data, scale fit trên tập nào, vì sao không leak.
- ☐ Kiến trúc baseline + lý do (số tầng/neuron/activation).
- ☐ Ảnh đường train/val loss + gọi tên hiện tượng (under/over/ổn).
- ☐ Chiến lược tuning (manual → random → Optuna) + chọn metric nào trên tập nào.
- ☐ Cấu hình thắng + bảng so sánh ≥ 3 cấu hình.
- ☐ Điểm test MỘT lần + F1/AUC/confusion matrix, không chỉ accuracy.

□ Một đoạn “nếu làm lại em sẽ thử gì khác và vì sao”.

Phụ lục — Đối chiếu framework

scikit-learn (nhập môn, ít boilerplate):

```
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import RandomizedSearchCV
clf = MLPClassifier(early_stopping=True, max_iter=300, random_state=SEED)
param_dist = {"hidden_layer_sizes": [(32,), (64,), (64,32), (128,64)],
              "alpha": [1e-4, 1e-3, 1e-2],
              "learning_rate_init": [1e-2, 1e-3, 1e-4],
              "batch_size": [32, 64, 128]}
search = RandomizedSearchCV(clf, param_dist, n_iter=15, cv=5,
                           scoring="roc_auc", random_state=SEED)
search.fit(X_train, y_train)
```

Đánh đổi: nhanh, có CV sẵn — nhưng **không thấy vòng lặp train**, kém về dạy “cơ chế”. Dùng để đối chứng. **Keras** ngắn gọn hơn nhưng cũng giấu vòng lặp. Với mục tiêu “SV giải thích từng bước”, **PyTorch (bản chính)** là lựa chọn thực hành tốt nhất vì vòng lặp rõ ràng.